



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Robles Chávez Uri Shuhua Alexei

N° de Cuenta: 320195318

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 19 de Septiembre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Ejercicio 1. Cohete espacial

Descripción

El primer ejercicio se terminó el modelo de el rover agregando su brazo articulado, pinza y seis llantas con rotaciones independientes.

Creación de primitivas geométricas

Para construir el rover se utilizaron las primitivas creadas previamente en el proyecto:

- Cubo.
- Pirámide triangular.
- Cilindro.
- Cono.
- Pirámide cuadrangular.

Estas figuras fueron almacenadas dentro de meshList, donde cada posición representa una geometría diferente. Estas primitivas posteriormente fueron reutilizadas para construir las diferentes partes del rover.

Uso de matrices para el modelado jerárquico

Una de las partes principales de la práctica fue implementar un sistema jerárquico mediante matrices auxiliares.

Se utilizaron tres matrices principales:

- glm::mat4 model;
- glm::mat4 modelaux;
- glm::mat4 modelaux2;

La matriz model representa la transformación actual del objeto que se está dibujando.

La matriz modelaux permite guardar una posición específica para continuar agregando componentes desde ese punto.

La matriz modelaux2 funciona como una referencia para crear ramas independientes, por ejemplo conectar las ruedas o el brazo directamente a la base del rover.

Construcción de la cabina principal

La primera pieza creada fue la cabina del rover.

Primero se trasladó el sistema de coordenadas hacia la posición donde se quería colocar el vehículo: `model = glm::translate(model,glm::vec3(0.0f,4.0f,-4.0f));`

Posteriormente se aplicó una transformación adicional y una escala: `glm::scale(model,glm::vec3(8.0f,4.0f,6.0f));`

Con esto se obtuvo el cuerpo principal utilizando un cubo escalado.

Construcción de la base inferior

La base del rover se creó utilizando la posición almacenada previamente en `modelaux`.

Esto permitió que la base permaneciera conectada con la cabina sin heredar transformaciones que no correspondían. Después se realizó la posición y escala:

- `glm::translate(model,glm::vec3(-1.0f,-2.75f,0.0f));`
- `glm::scale(model,glm::vec3(10.0f,1.5f,8.0f));`

```
333 //NOS POSICIONAMOS EN EL CENTRO DE LA BASE
334 model = glm::translate(model, glm::vec3(-1.0f, -2.75f, 0.0f));
335 modelaux = model; //guardamos la matriz de modelo para conectar los siguientes objetos a la base
336 model = glm::scale(model, glm::vec3(10.0f, 1.5f, 8.0f));
337 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
338 color = glm::vec3(0.45f, 0.45f, 0.45f);
339 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
340 meshList[0]->RenderMesh();
341
342 model = modelaux;
343 modelaux2 = model; //guardamos la matriz de modelo para conectar los siguientes objetos a la base
344 // modelaux2 conserva la base para empezar cada rama.
345
346 //BASE PARA EL BRAZO
347 model = modelaux2;
348 model = glm::translate(model, glm::vec3(-3.5f, 1.25f, 2.5f));
349 //modelaux = model; //guardamos la matriz de modelo para conectar los siguientes objetos a la base
350 //model = glm::scale(model, glm::vec3(10.0f, 1.5f, 8.0f)); NO se requirio escalar el cubo
351 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
352 color = glm::vec3(0.75f, 0.75f, 0.75f);
353 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
354 meshList[0]->RenderMesh();
```

Figura 1. Base inferior conectada con la cabina.

Creación del brazo articulado

El brazo fue uno de los elementos principales del modelo jerárquico.

Primero se creó una base conectada a la estructura principal y posteriormente se agregó un punto de rotación. Esto permitió controlar el movimiento del primer segmento del brazo. El brazo fue construido utilizando cilindros escalados:

```

366 //Brazo
367 model = modelaux;
368 model = glm::translate(model, glm::vec3(-1.06066f, 1.06066f, 0.0f));
369 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
370 modelaux = model;
371 model = glm::scale(model, glm::vec3(0.5f, 3.0f, 0.5f));
372 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
373 color = glm::vec3(0.75f, 0.75f, 0.75f);
374 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
375 meshList[2]->RenderMesh();
376
377 //eje br1-br22
378 model = modelaux;
379 model = glm::translate(model, glm::vec3(0.0f, 1.5f, 0.0f));
380 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
381 modelaux = model;
382 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
383 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
384 color = glm::vec3(0.0f, 0.0f, 1.0f);
385 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
386 sp.render();

```

Figura 2. Segmentos del brazo utilizando transformaciones jerárquicas.

Muñeca y articulaciones

Después del segundo segmento del brazo se agregó la muñeca como un punto de unión. Esta parte utiliza una esfera para representar la articulación y se controla mediante: `mainWindow.getarticulacion3()`.

La utilización de puntos de articulación permitió que cada parte pudiera rotar conservando la conexión con la anterior.

```

399 //muñeca. H gira la pinza desde el extremo del segundo brazo.
400 model = modelaux;
401 model = glm::translate(model, glm::vec3(0.0f, 2.0f, 0.0f));
402 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
403 modelaux = model;
404 model = glm::scale(model, glm::vec3(0.35f, 0.35f, 0.35f));
405 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
406 color = glm::vec3(0.0f, 0.0f, 1.0f);
407 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
408 sp.render();
409

```

Figura 3. Articulación de la muñeca mediante una esfera.

Construcción de la pinza

La pinza fue construida mediante una base y dos dedos móviles.

La base utiliza un cubo escalado: `glm::scale(model, glm::vec3(0.9f, 0.3f, 0.4f))`;

Para controlar la apertura se calculó un ángulo: `mainWindow.getarticulacion4()` y los dos dedos se generan mediante un ciclo: `for(int lado=-1; lado<=1; lado+=2)`. Esto permitió que ambos lados de la pinza se movieran de forma simétrica.

```

410 // Base de la pinza.
411 model = modelaux;
412 model = glm::translate(model, glm::vec3(0.0f, 0.35f, 0.0f));
413 modelaux = model;
414 model = glm::scale(model, glm::vec3(0.9f, 0.3f, 0.4f));
415 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
416 color = glm::vec3(0.4f, 0.4f, 0.4f);
417 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
418 meshList[0]->RenderMesh();
419
420 // J abre y cierra la pinza entre 0 y 35 grados.
421 GLfloat apertura = fmod(mainWindow.getarticulacion4(), 140.0f);
422 if (apertura > 70.0f) apertura = 140.0f - apertura;
423 apertura = apertura / 2.0f;
424 glm::mat4 modelaux3(1.0f);
425
426 // Un dedo a cada lado y los dos parten de la base de la pinza.
427 for (int lado = -1; lado <= 1; lado += 2)
428 {
429     model = modelaux;
430     model = glm::translate(model, glm::vec3(lado * 0.35f, 0.15f, 0.0f));
431     model = glm::rotate(model, glm::radians(-lado * apertura), glm::vec3(0.0f, 0.0f, 1.0f));
432     modelaux3 = model;
433     model = glm::translate(model, glm::vec3(0.0f, 0.425f, 0.0f));
434     model = glm::scale(model, glm::vec3(0.18f, 0.85f, 0.25f));
435     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
436     color = glm::vec3(0.8f, 0.8f, 0.8f);
437     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
438     meshList[0]->RenderMesh();
439
440 //punta doblada hacia el centro.
441 model = modelaux3;
442 model = glm::translate(model, glm::vec3(-lado * 0.15f, 0.85f, 0.0f));
443 model = glm::scale(model, glm::vec3(0.4f, 0.18f, 0.25f));
444 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
445 meshList[0]->RenderMesh();
446 }

```

Figura 4. Pinza del rover con movimiento mediante transformación jerárquica.

Construcción de las ruedas

Una de las modificaciones más importantes fue completar las seis llantas del rover.

Se definieron tres posiciones diferentes para cada lado: GLfloat posicionX[] = {-7.5f, -0.5f, 6.0f}; De esta manera se generan tres ruedas cercanas y tres ruedas lejanas.

Cada rueda tiene soportes conectados a la base y barras inclinadas que llegan hasta el centro de la llanta.

Además, cada una utiliza una articulación diferente para que pueda girarse de forma independiente, en el código estas corresponden a las teclas K, L, U, I, O y P.

```

473     for (int barra = -1; barra <= 1; barra += 2)
474     {
475         model = modelaux2;
476         model = glm::translate(model, glm::vec3(
477             (inicioX[rueda] + posicionX[rueda]) / 2.0f, -1.125f,
478             lado * soporteZ[rueda] + barra * 0.14f));
479         model = glm::rotate(model, inclinacion, glm::vec3(0.0f, 0.0f, 1.0f));
480         model = glm::scale(model, glm::vec3(longitud, 0.28f, 0.14f));
481         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
482         color = glm::vec3(0.78f, 0.78f, 0.72f);
483         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
484         meshList[0]-->RenderMesh();
485     }
486
487     // eje entre las barras y el centro de la llanta.
488     model = modelaux2;
489     model = glm::translate(model, glm::vec3(posicionX[rueda], -2.0f,
490         lado * (soporteZ[rueda] + llantaZ[rueda]) / 2.0f));
491     model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
492     model = glm::scale(model, glm::vec3(0.22f,
493         llantaZ[rueda] - soporteZ[rueda] + 0.45f, 0.22f));
494     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
495     color = glm::vec3(0.45f, 0.45f, 0.45f);
496     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
497     meshList[2]-->RenderMesh();
498
499     GLfloat giro = 0.0f;
500     if (rueda == 0) {
501         if (lado == 1) giro = mainWindow.getarticulacion5(); // K
502         else giro = mainWindow.getarticulacion6(); // L
503     }
504     if (rueda == 1) {
505         if (lado == 1) giro = mainWindow.getarticulacion7(); // U
506         else giro = mainWindow.getarticulacion8(); // I
507     }
508     if (rueda == 2) {
509         if (lado == 1) giro = mainWindow.getarticulacion9(); // O
510         else giro = mainWindow.getarticulacion10(); // P
511     }
512
513     //ninguna rueda hereda el giro de otra.
514     model = modelaux2;
515     model = glm::translate(model, glm::vec3(posicionX[rueda], -2.0f,
516         lado * llantaZ[rueda]));
517     model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 0.0f, 1.0f));
518     modelaux3 = model;
519     model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
520     model = glm::scale(model, glm::vec3(1.25f, 0.7f, 1.25f));
521     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
522     color = glm::vec3(0.04f, 0.04f, 0.04f);
523     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
524     meshList[2]-->RenderMesh();
525
526     //rayos en ambas caras
527     for (int cara = -1; cara <= 1; cara += 2)
528     {
529         for (int rayo = 0; rayo < 4; rayo++)
530         {
531             model = modelaux3;
532             model = glm::translate(model, glm::vec3(0.0f, 0.0f, cara * 0.37f));
533             model = glm::rotate(model, glm::radians(rayo * 45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
534             model = glm::scale(model, glm::vec3(2.1f, 0.12f, 0.08f));
535             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
536             color = glm::vec3(0.7f, 0.7f, 0.7f);
537             glUniform3fv(uniformColor, 1, glm::value_ptr(color));
538             meshList[0]-->RenderMesh();
539         }
540     }
541 }

```

Figura 5. Ruedas con rotación independiente y detalles internos.

Resultado

Al finalizar el primer ejercicio se obtuvo un rover tridimensional formado por una cabina, una base, un brazo mecánico con varias articulaciones, una pinza y seis ruedas.

El uso de modelado jerárquico permitió que las piezas permanecieran conectadas mientras se modificaban las articulaciones. El brazo puede moverse por secciones, la pinza puede abrirse y cerrarse y cada una de las seis ruedas puede girar de manera independiente.

La utilización de matrices auxiliares permitió regresar a puntos específicos de la estructura sin perder las transformaciones de las piezas principales.

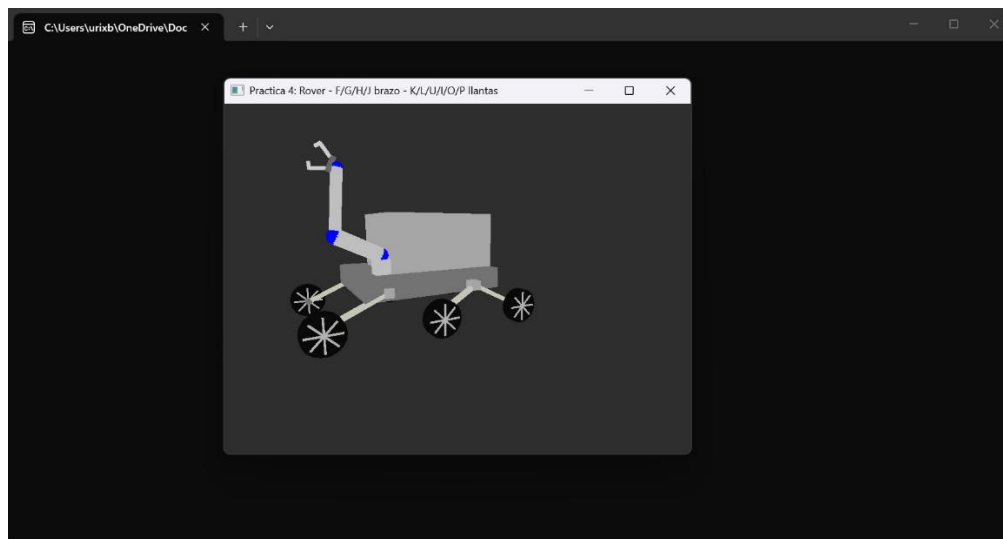


Figura 6. Resultado final del rover con cabina, brazo, pinza y seis llantas.

Ejercicio 2. Sonda espacial jerárquica

Descripción

En el segundo se construyó una sonda espacial de manera jerárquica, utilizando un cuerpo central, paneles solares articulados, una antena y un sistema de propulsión. Para ambos ejercicios se reutilizaron primitivas geométricas como cubos, cilindros, conos, pirámides y esferas.

Matriz padre de la sonda

Para la sonda se creó primero una matriz principal que funciona como padre de todas sus partes.

Esta matriz recibe las rotaciones globales: `mainWindow.getrotax()`, `mainWindow.getrotay()`, `mainWindow.getrotaz()`.

De esta manera, al girar la sonda completa, todas las piezas conectadas también se mueven.

Las teclas E, R y T permiten modificar estas rotaciones globales. Esto también se encuentra indicado al inicio del archivo.

```
551 // matriz padre: todas las piezas heredan el giro de la sonda.  
552 model = glm::mat4(1.0f);  
553 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1, 0, 0));  
554 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0, 1, 0));  
555 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0, 0, 1));  
556 modelaux = model; // Guardamos antes del escalado del cuerpo.
```

Figura 7. Matriz principal utilizada para controlar todo el modelo de la sonda.

Cuerpo central

El cuerpo de la sonda se realizó mediante un cubo de color dorado.

Se escaló de la siguiente manera: `glm::vec3(3.0f,3.0f,2.0f)`.

Este cuerpo funciona como la pieza principal de la jerarquía, ya que los paneles solares, la antena y el propulsor se colocan tomando como referencia su matriz.

```
561 //cuerpo central: un cubo dorado.  
562 model = glm::scale(model, glm::vec3(3.0f, 3.0f, 2.0f));  
563 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
564 color = glm::vec3(0.8f, 0.6f, 0.15f);  
565 glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
566 meshList[0]->RenderMesh();
```

Figura 8. Cubo escalado utilizado como cuerpo principal de la sonda espacial.

Paneles solares

Se colocaron dos paneles solares, uno a cada lado del cuerpo.

Para evitar duplicar código se utilizó: `for (int lado = -1; lado <= 1; lado += 2)`

Cada panel tiene su propia articulación, por lo tanto, el panel izquierdo y el panel derecho pueden girar de manera independiente.

Las teclas F y G se utilizan para estas articulaciones.

Cada panel está conectado mediante un soporte y se construye utilizando un cubo muy delgado y escalado.

Además, se agregaron tres divisiones para representar visualmente las celdas solares.

```
568 // paneles solares
569 for (int lado = -1; lado <= 1; lado += 2)
570 {
571     model = modelaux;
572     model = glm::translate(model, glm::vec3(lado * 1.5f, 0.0f, 0.0f));
573     GLfloat giro = (lado == -1) ? mainWindow.getarticulacion1() : mainWindow.getarticulacion2();
574     model = glm::rotate(model, glm::radians(lado * giro), glm::vec3(0, 0, 1));
575     modelaux2 = model; // Bisagra: F mueve el panel izquierdo; G el derecho.
576
577     //soporte entre el cuerpo y el panel
578     model = glm::translate(model, glm::vec3(lado * 0.5f, 0.0f, 0.0f));
579     model = glm::scale(model, glm::vec3(1.0f, 0.2f, 0.2f));
580     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
581     color = glm::vec3(0.7f, 0.7f, 0.7f);
582     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
583     meshList[0] -> RenderMesh();
584
585     // panel delgado, conectado al extremo del soporte.
586     model = modelaux2;
587     model = glm::translate(model, glm::vec3(lado * 3.0f, 0.0f, 0.0f));
588     modelaux2 = model; // Las divisiones tambien se mueven con el panel.
589     model = glm::scale(model, glm::vec3(4.0f, 0.16f, 3.0f));
590     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
591     color = glm::vec3(0.05f, 0.2f, 0.65f);
592     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
593     meshList[0] -> RenderMesh();
594
595     // tres divisiones para representar las celdas solares.
596     for (int division = -1; division <= 1; division++)
597     {
598         model = modelaux2;
599         model = glm::translate(model, glm::vec3(division * 1.0f, 0.0f, 0.0f));
600         model = glm::scale(model, glm::vec3(0.05f, 0.18f, 3.0f));
601         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
602         color = glm::vec3(0.55f, 0.65f, 0.85f);
603         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
604         meshList[0] -> RenderMesh();
605     }
606 }
```

Figura 9. Paneles solares conectados jerárquicamente al cuerpo de la sonda.

Antena

La antena se colocó sobre la parte superior del cuerpo principal.

Cuenta con dos movimientos: `mainWindow.getarticulacion3()` para modificar su orientación y `mainWindow.getarticulacion4()` para modificar la inclinación.

En el programa estas articulaciones están relacionadas con las teclas H y J.

La antena está formada por un mástil cilíndrico, un plato simplificado mediante una esfera aplanada y un receptor central.

```

608 // antena
609 model = modelaux;
610 model = glm::translate(model, glm::vec3(0.0f, 1.5f, 0.0f));
611 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0, 1, 0));
612 model = glm::rotate(model, glm::radians(25.0f + mainWindow.getarticulacion4()), glm::vec3(0, 0, 1));
613 modelaux2 = model; // H orienta la antena; J cambia su inclinacion.

615 //mastil cilindrico
616 model = glm::translate(model, glm::vec3(0.0f, 0.6f, 0.0f));
617 model = glm::scale(model, glm::vec3(0.12f, 1.2f, 0.12f));
618 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
619 color = glm::vec3(0.7f, 0.7f, 0.7f);
620 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
621 meshList[2]->RenderMesh();
622
623 // plato simplificado
624 model = modelaux2;
625 model = glm::translate(model, glm::vec3(0.0f, 1.4f, 0.0f));
626 model = glm::scale(model, glm::vec3(1.2f, 0.2f, 1.2f));
627 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
628 color = glm::vec3(0.9f, 0.9f, 0.9f);
629 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
630 sp.render();
631
632 // receptor en el centro del plato.
633 model = modelaux2;
634 model = glm::translate(model, glm::vec3(0.0f, 1.9f, 0.0f));
635 model = glm::scale(model, glm::vec3(0.07f, 0.6f, 0.07f));
636 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
637 color = glm::vec3(0.9f, 0.65f, 0.15f);
638 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
639 meshList[2]->RenderMesh();

```

Figura 10. Antena jerárquica formada por mástil, plato y receptor.

Sistema de propulsión

En la parte inferior se agregó un sistema de propulsión.

Primero se utiliza un cilindro gris y después se coloca un cono debajo para completar la forma del propulsor. El cono utiliza RenderMeshGeometry() debido a la forma en que está construida esta geometría.

```

641 // propulsor
642 model = modelaux;
643 model = glm::translate(model, glm::vec3(0.0f, -1.75f, 0.0f));
644 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
645 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
646 color = glm::vec3(0.35f, 0.35f, 0.35f);
647 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
648 meshList[2]->RenderMesh();

650 model = modelaux;
651 model = glm::translate(model, glm::vec3(0.0f, -2.35f, 0.0f));
652 model = glm::scale(model, glm::vec3(0.4f, 1.0f, 0.4f));
653 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
654 color = glm::vec3(0.55f, 0.55f, 0.55f);
655 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
656 meshList[3]->RenderMeshGeometry(); // El cono utiliza GL_TRIANGLE_FAN.

```

Figura 11. Cilindro y cono utilizados para formar el sistema de propulsión.

Resultado

Al finalizar el segundo ejercicio se obtuvo una sonda espacial tridimensional formada jerárquicamente.

El modelo incluye un cuerpo central dorado, dos paneles solares articulados, una antena móvil y un sistema de propulsión.

Toda la sonda puede rotarse utilizando las teclas E, R y T. Los paneles pueden modificarse de manera independiente mediante F y G, mientras que la orientación e inclinación de la antena se controlan mediante H y J. Además, la cámara continúa utilizando WASD y el mouse.

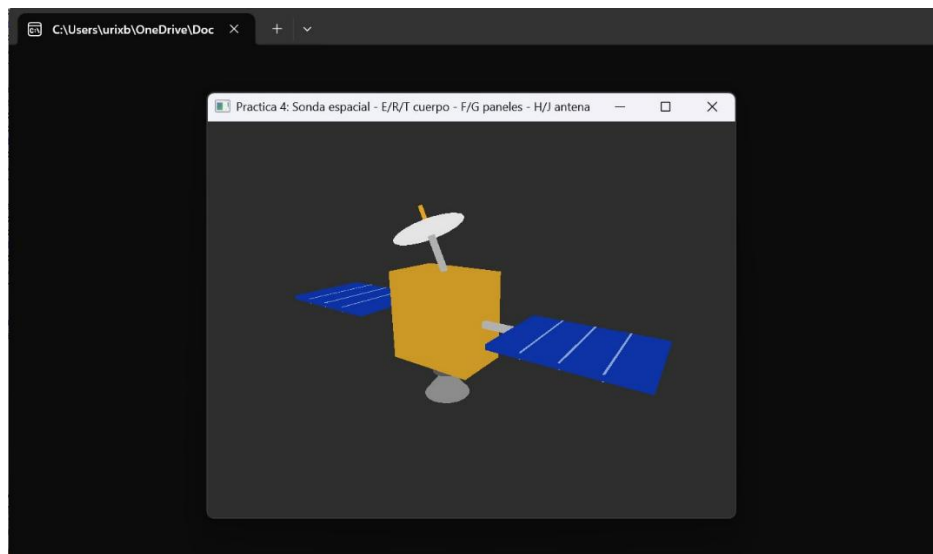


Figura 12. Resultado final de la sonda espacial construida mediante modelado jerárquico.

Problemas presentados

En el ejercicio del rover, uno de los principales problemas que tuve fue completar las seis ruedas sin perder la relación jerárquica con la base. Cada rueda debía contar con su propio soporte, mantener una posición específica y utilizar una articulación diferente para poder girar de forma independiente.

También fue necesario que manejara correctamente las matrices auxiliares para evitar que el movimiento de una rueda afectara accidentalmente a las demás.

En el ejercicio de la sonda espacial, la principal dificultad que tuve fue organizar la jerarquía para que todas las piezas heredaran la rotación general del cuerpo, pero

al mismo tiempo los paneles solares y la antena conservaran movimientos independientes.

También tuve que ajustar posiciones y escalas para conseguir que las piezas permanecieran visualmente conectadas al cuerpo central.

Conclusión:

- a) La complejidad de estos ejercicios fue mayor que en prácticas anteriores debido a que se trabajó con varias jerarquías dentro de un mismo modelo. En el rover la implementación de seis ruedas independientes aumentó la dificultad, ya que cada una debía tener su propio valor de rotación sin afectar las demás. En la sonda, la dificultad principal consistió en mantener una transformación global para todo el objeto y agregar dentro de ella transformaciones locales para los paneles y la antena.
- b) La práctica me permitió comprender de manera más clara cómo funciona el modelado jerárquico dentro de una escena tridimensional, los dos ejercicios me permitieron aplicar de una forma más completa los conceptos de traslación, rotación, escala y herencia de transformaciones.
- c) Conclusión: Al finalizar esta práctica se concluye que el modelado jerárquico permite construir objetos tridimensionales complejos mediante la relación entre componentes padres e hijos. En el rover fue posible integrar una cabina, base, brazo articulado, pinza y seis ruedas independientes utilizando matrices auxiliares para conservar correctamente las transformaciones. En la sonda espacial se utilizó una transformación general para controlar todo el modelo y transformaciones locales para permitir el movimiento independiente de los paneles solares y de la antena. Con esto se resaltó la importancia de almacenar y reutilizar matrices de transformación, así como la manera en que una estructura jerárquica facilita el movimiento y organización de modelos tridimensionales compuestos por varias piezas.

Bibliografía

- BorrowBits. (2013, 7 de julio). *Diseño e implementación de un modelo jerárquico: Robot. [Código incluido]*. BorrowBits. <https://openglgraficacion.blogspot.com/2016/02/trasformaciones-geometricas.html>
- Osorno, C. (2016, 29 de febrero). *Transformaciones geométricas en OpenGL*. Graficación OpenGL. <https://openglgraficacion.blogspot.com/2016/02/trasformaciones-geometricas.html>